Security Assessment of OWASP Juice Shop

Oct 17, 2026 · Prepared by @hi

Executive summary

This assessment examined a locally deployed instance of OWASP Juice Shop, a deliberately vulnerable e-commerce web application used for security training. Testing identified 8 distinct security weaknesses, including one critical flaw that allows an attacker to log in as the site administrator without knowing any password, and a related flaw that exposes every customer's email address and lets an outside party delete customer feedback. Other issues found include a way to run malicious code in a victim's browser, weak protection of stored passwords, and a login page with no defense against automated password-guessing. If this were a production system, these issues could lead to full compromise of customer accounts and administrative control of the store. All findings include step-by-step reproduction evidence and prioritized, actionable fixes.

Scope and methodology

Scope: A single instance of OWASP Juice Shop (image `bkimminich/juice-shop:latest`), deployed locally via Docker (`docker run -d -p 3000:3000 bkimminich/juice-shop`) and reachable only at `http://localhost:3000`. No external or production systems were in scope.

Approach: Grey-box manual testing, performed as an authenticated low-privilege customer and, where applicable, as an unauthenticated visitor. Testing followed the OWASP Testing Guide and the OWASP Top 10 (2021) category structure, prioritizing injection, broken access control, cryptographic failures, identification and authentication failures, and security misconfiguration.

Tools used:

- Chrome DevTools (Network and Console tabs) for request/response inspection and manual API calls
- Manual payload crafting for SQL injection and XSS test cases
- Browser-based manual testing of authentication, session, and access-control flows

Out of scope: Automated vulnerability scanning, denial-of-service testing, and any testing against infrastructure outside the local container.

Duration: Testing was conducted in a single session on 17 October 2026.

Findings summary

#	Finding	Severity	CVSS 3.1	Description
1	SQL Injection - Authentication Bypass	Critical	9.1	Login form allows bypassing authentication and logging in as any user, including admin, via SQL injection.
2	Broken Access Control - Admin Panel	High	8.0	Any authenticated user can reach the admin panel, view all user emails, and delete other users' feedback.
3	DOM-based Cross-Site Scripting	Medium	6.1	The search field renders unsanitized input, allowing arbitrary JavaScript execution in a victim's browser.
4	Security Misconfiguration - Version Disclosure	Medium	5.3	Error pages disclose the exact backend framework and version, aiding targeted exploitation.
5	Insecure Direct Object Reference - Basket	Medium	6.4	A valid user's token can retrieve other users' shopping basket contents by changing a numeric ID.
6	Weak Cryptographic Hash for Passwords	Medium	5.3	Passwords are hashed with MD5, a broken algorithm vulnerable to fast offline cracking.
7	Insecure Password Recovery Mechanism	High	7.4	Password reset relies on guessable/researchable security questions, enabling account takeover.
8	Missing Rate Limiting on Authentication	Medium	5.3	The login endpoint has no lockout, delay, or CAPTCHA, enabling unlimited automated password guessing.

Detailed Findings

Finding 1: SQL Injection - Authentication Bypass

Severity: Critical **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N` (Base score 9.1)

Description

The login form builds a SQL query by directly concatenating the user-supplied email field into the query string, without parameterization or input sanitization. An attacker can inject SQL syntax into the email field to alter the query's logic and bypass the password check entirely.

Steps to reproduce

1. Navigate to `http://localhost:3000/#/login`.
2. In the Email field, enter: `' OR 1=1--`
3. In the Password field, enter any value (e.g. `x`).
4. Click Log in.

Result: The application returns HTTP 200 with a valid session token and logs the attacker in as the first user in the database (the administrator account), with no valid credentials supplied. The application itself confirms this as the "Login Admin" challenge.

A second variant targets a specific known user instead of defaulting to the first row:

1. Email: `jim@juice-sh.op'--`
2. Password: any value

Result: Logs in directly as `jim@juice-sh.op`, confirmed by the application as the "Login Jim" challenge. This shows the flaw can be used to take over any account whose email address is known, not only the first row in the table.

Business impact

An attacker with no prior credentials can gain full administrative control of the store, or take over any specific customer account by email address alone. This could lead to theft of customer data, fraudulent orders, defacement of the storefront, and complete loss of trust in the platform's security.

Remediation

- Replace string-concatenated SQL queries with parameterized queries / prepared statements for all database access, especially authentication.
- Use an ORM (e.g. Sequelize, already present in the codebase) consistently and avoid raw SQL string building for user-controlled input.
- Apply strict input validation on the email field format before it reaches the query layer.
- Add automated tests (e.g. SAST/DAST in CI) that specifically probe authentication endpoints for injection.

Finding 2: Broken Access Control - Admin Panel Exposure

Severity: High **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N` (Base score 8.0)

Description

The `/#/administration` route and its underlying API do not verify that the requesting session belongs to a user with administrative privileges. Any authenticated session that can reach this route is able to view the full list of registered users (including email addresses) and moderate (delete) customer feedback.

Steps to reproduce

1. Authenticate as any user (including via the Finding 1 bypass).
2. Navigate directly to `http://localhost:3000/#/administration`.
3. Observe the "Registered Users" panel listing every account's email address.
4. In the "Customer Feedback" panel, click the trash icon on an entry not belonging to the current user.

Result: The application confirms the "Admin Section" challenge as solved. All registered emails are visible, and the targeted feedback entry is successfully deleted (confirmed by it becoming greyed out / disabled immediately after the action), proving unauthorized data modification, not just viewing.

Business impact

Exposes the full customer email list, usable for phishing or credential-stuffing campaigns. The ability to delete other users' content without authorization undermines data integrity and could be used to suppress negative reviews or disrupt business operations.

Remediation

- Enforce server-side role-based access control (RBAC) on every admin route and API endpoint, checking the authenticated user's role, not just the presence of a valid token.
- Return HTTP 403 Forbidden for non-admin sessions attempting to access admin-only resources.
- Apply the same server-side check to the underlying REST endpoints the admin page calls, not only the frontend route.

Finding 3: DOM-based Cross-Site Scripting (XSS)

Severity: Medium **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N` (Base score 6.1)

Description

The product search feature takes the `q` query parameter and renders it into the page's DOM without sanitization or output encoding. An attacker can inject an HTML/JavaScript payload that executes in the context of any user who opens a crafted link.

Steps to reproduce

1. Navigate to the search feature at `http://localhost:3000/#/search`.
2. Enter the following payload into the search box: `<iframe src="javascript:alert('xss')">`
3. Press Enter / submit the search.

Result: A JavaScript alert box fires immediately, proving arbitrary script execution. The full payload is reflected in the URL (`/#/search?q=<iframe src="javascript:alert('xss')">`), meaning the exploit can be delivered as a single link to a victim. The application confirms this as the "DOM XSS" challenge.

Business impact

An attacker could craft a malicious link and distribute it via email or social media; any victim who clicks it while logged in could have their session cookie stolen, be redirected to a phishing page, or have actions performed on their behalf (e.g. placing orders, changing account details).

Remediation

- Sanitize and HTML-encode all user-supplied input before rendering it into the DOM.
- Use the framework's built-in safe-binding mechanisms (e.g. Angular's `DomSanitizer`) rather than directly injecting raw HTML.
- Implement a strict Content Security Policy (CSP) header to block inline script execution and restrict `javascript:` URIs as a defense-in-depth measure.

Finding 4: Security Misconfiguration - Verbose Error Messages / Version Disclosure

Severity: Medium **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N` (Base score 5.3)

Description

When an unauthenticated or malformed request is sent to a protected REST endpoint, the application returns a default Express error page that discloses the exact backend framework and version number in the page title.

Steps to reproduce

1. In a browser with no active session, navigate directly to `http://localhost:3000/rest/basket/1`.

2. Observe the returned error page.

Result: The page displays the heading "OWASP Juice Shop (Express ^4.22.1)" alongside a 401 UnauthorizedError. The application itself confirms this as the "Error Handling" challenge (provoking an error that is not gracefully or consistently handled).

Business impact

Knowing the exact backend framework and version allows an attacker to search for publicly known vulnerabilities (CVEs) specific to that version, significantly narrowing the effort needed to find a working exploit, and signals that error handling practices may be inconsistent elsewhere in the application.

Remediation

- Configure custom, generic error pages for all HTTP error codes that do not reveal framework, library, or version information.
- Disable Express's default error handler in production (`NODE_ENV=production` and a custom error middleware).
- Centralize error handling so all endpoints return consistent, minimal error responses to clients while logging full details only server-side.

Finding 5: Insecure Direct Object Reference (IDOR) - Shopping Basket

Severity: Medium **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` (Base score 6.4)

Description

The `/rest/basket/{id}` endpoint checks only that the request carries a valid authentication token; it does not verify that the basket ID requested actually belongs to the authenticated user. Any logged-in user can view any other user's basket simply by changing the numeric ID in the request.

Steps to reproduce

1. Log in as a test user (e.g. `jim@juice-sh.op`) and note your own basket ID from the Network tab (basket ID 2 in this case).
2. In the browser console, run: `fetch('/rest/basket/1', {headers: {Authorization: 'Bearer <your token>'}}).then(r=>r.json()).then(console.log)`
3. Observe the response.

Result: The request returns `{status: 'success', data: {...}}` containing basket ID 1's contents, a different `UserId` than the authenticated account. The application confirms this as the "View Basket" challenge (viewing another user's shopping basket).

Business impact

Exposes other customers' shopping activity and basket contents. If basket write operations are similarly unchecked, this could extend to tampering with another user's order before checkout, affecting billing integrity.

Remediation

- Add server-side ownership checks on every object-returning endpoint: verify the authenticated user's ID matches the resource's owning user ID before returning data.
- Use indirect, unguessable reference identifiers (e.g. UUIDs) instead of small sequential integers where feasible, as defense in depth.
- Apply the same ownership check consistently to read, update, and delete operations on the basket resource.

Finding 6: Use of a Weak Cryptographic Hash Function (MD5) for Password Storage

Severity: Medium **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N` (Base score 5.3)

Description

The JSON Web Token (JWT) issued on login embeds the user's password hash directly in its payload, and the hash is computed with MD5, a cryptographic hash function that is fast to compute and long known to be broken for password storage (vulnerable to rainbow-table and brute-force attacks).

Steps to reproduce

1. Log in as any user and capture the issued JWT from the Authorization header of any authenticated request.
2. Decode the JWT payload (e.g. using jwt.io or a manual Base64 decode of the middle segment).
3. Inspect the `password` field in the decoded payload.

Result: The decoded payload contains a 32-character hexadecimal string (e.g. `e541ca7ecmf72b8d1286474fc613e5e45`) consistent with an MD5 hash, exposed to the client on every authenticated request.

Business impact

MD5 hashes can be cracked in seconds to minutes using modern hardware and precomputed rainbow tables, especially for common or short passwords. Embedding the hash in a client-visible token further increases exposure, as any party able to intercept or inspect the token

(e.g. via a separate XSS flaw, see Finding 3) gains direct access to crackable password material.

Remediation

- Migrate password storage to a modern, slow, salted hashing algorithm designed for passwords, such as bcrypt, scrypt, or Argon2.
- Never include password hashes, even strong ones, inside client-visible tokens such as JWTs; keep them server-side only.
- Force a password reset for all existing users once the hashing scheme is upgraded.

Finding 7: Insecure Password Recovery Mechanism

Severity: High **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N` (Base score 7.4)

Description

The "Forgot Password" flow at `/#/forgot-password` allows a password reset based solely on correctly answering a pre-set security question (e.g. mother's maiden name, first pet). Security questions of this kind are frequently guessable, researchable via social media/public records, or reused across services, making them a weak substitute for identity verification.

Steps to reproduce

1. Navigate to `http://localhost:3000/#/forgot-password`.
2. Enter a known user's email address.
3. Observe that the form requests only the answer to a security question before allowing a new password to be set, with no secondary verification (e.g. email confirmation link, OTP).

Result: An attacker who can correctly guess or research the answer to a target's security question can set a new password for that account without ever knowing the original password.

Business impact

Account takeover via guessed or researched security answers is a well-documented real-world attack pattern. Combined with publicly available personal information (common on social media), this significantly lowers the bar for targeted account compromise, especially for high-value accounts such as administrators.

Remediation

- Replace or supplement security questions with a time-limited, single-use reset link or code sent to the user's verified email or phone number.
- If security questions must be retained, require them in combination with another verified factor, never as the sole check.
- Rate-limit and log password reset attempts per account to detect brute-forcing of security answers.

Finding 8: Missing Rate Limiting on Authentication Endpoint

Severity: Medium **CVSS 3.1 vector:** `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N` (Base score 5.3)

Description

The `/rest/user/login` endpoint does not implement account lockout, exponential back-off, or CAPTCHA after repeated failed login attempts for the same account, allowing unlimited automated credential-guessing attempts.

Steps to reproduce

1. Navigate to `http://localhost:3000/#/login`.
2. Submit an incorrect password for the same account (e.g. `admin@juice-sh.op`) 5-6 times in rapid succession.
3. Observe the application's response and timing on each attempt.

Result: Every attempt returns an identical, instant "Invalid email or password" response with no increasing delay, lockout message, or CAPTCHA challenge, confirmed via repeated HTTP 401 responses in the Network tab with no throttling between them.

Business impact

Combined with weak or reused passwords, this allows attackers to run automated credential-stuffing or brute-force attacks against any account, including the administrator account, without triggering any defensive control.

Remediation

- Implement account lockout or exponential back-off after a small number of failed login attempts (e.g. 5), with lockout notifications to the account owner.
- Add a CAPTCHA or equivalent challenge after repeated failures from the same IP or against the same account.
- Apply IP-based and account-based rate limiting at the API gateway or application middleware layer.

Conclusion and overall recommendations

This assessment found 8 vulnerabilities spanning injection, broken access control, cross-site scripting, cryptographic weaknesses, and missing authentication safeguards. The most urgent issue (Finding 1) allows unauthenticated attackers to gain full administrative access, and should be remediated immediately before any other hardening work. Findings 2, 5, and 7 compound this risk by exposing further data and account-takeover paths once any foothold is gained.

Priority order for remediation:

1. Finding 1 - SQL Injection (Critical) - fix immediately
2. Finding 7 - Insecure Password Recovery (High)
3. Finding 2 - Broken Access Control (High)
4. Findings 3, 5, 6, 8 (Medium) - address in the next development cycle
5. Finding 4 (Medium, lowest effort) - fix alongside general error-handling cleanup

Adopting parameterized queries, consistent server-side authorization checks, output encoding, and modern password hashing would resolve the majority of these findings and meaningfully raise the application's overall security posture.